AZURE MEETUP PUGLIA

BY COMMUNITY • FOR COMMUNITY

#GLOBALAZURE

2026

BENVENUTI

GLOBAL AZURE PUGLIA 2026

Venerdì 17 aprile - Bari

Grazie Agli Sponsor

AZURE MEETUP PUGLIA

Sponsor Gold

UGOLOPEZ.IT

Sponsor Silver

CODE architects — Tomorrow's software, today

eusoft — more than a ums

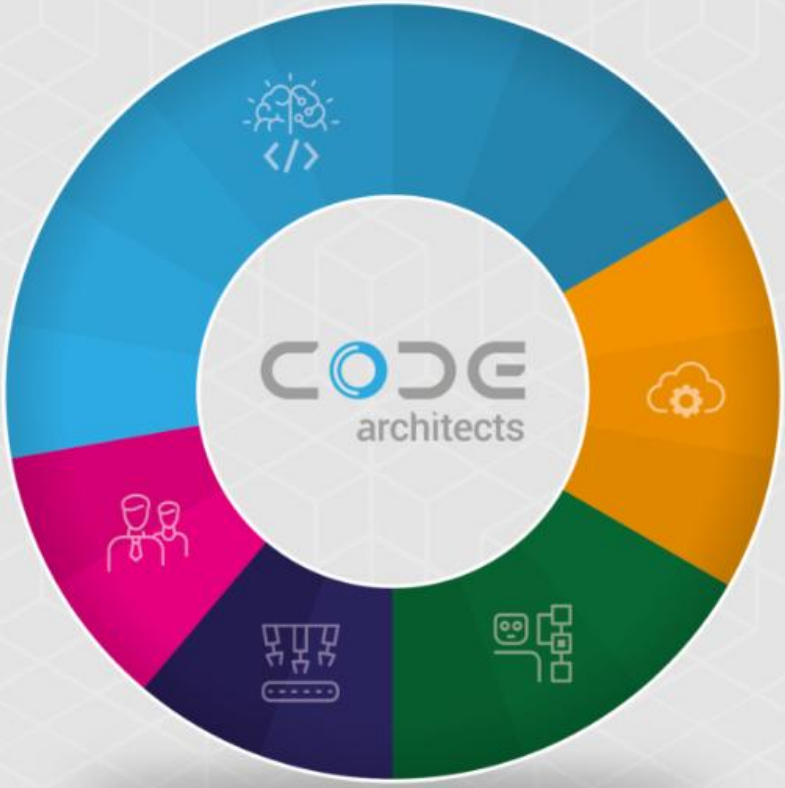
Build AI-powered solutions using SQL Server 2025

- SQL Server 2025 news
- A new way of search
- JSON
- REG_EXP

pceglie@codearchitects.com

Sviluppo Software

- Applicazioni enterprise per il cloud
- Database, big data e Business Intelligence
- Migrazione di applicazioni Visual Basic
- Tool per developer
- DevOps
- Team as a Service
- UX/UI: User eXperience
- RPA: Robotic Process Automation



Produttività, reti e infrastrutture

- Cloud migration & management
- Cyber-security & business continuity
- Productivity & smart working

Consulting

- Formazione & Mentoring
- Ricerca & Sviluppo

Meccatronica

- Progettazione macchine speciali
- Veicoli industriali e robot mobili custom

Automazione industriale e gestione della produzione

- Controllo di macchine e linee di produzione
- Sistemi di monitoraggio real-time
- Ottimizzazione processi e produzione

Generally Available

SQL Server 2025

The AI-ready enterprise database from ground to cloud

AI built-in | Built for developers | Mission Critical

aka.ms/sqlserver2025

Building on a foundation of innovation

SQL Server 2017

SQL Server on Linux

Containers

Adaptive query processing

Automatic Tuning

Graph database

Machine learning services

SQL Server 2019

Data virtualization

Intelligent query processing

Accelerated database recovery

Data classification

SQL Server 2022

Cloud Connected

IQP NextGen

Ledger

Data Lakes

T-SQL enhancements

SQL platform choices

Azure SQL Database

- Fully managed PaaS with 99.99% SLA
- Hyperscale tier - limitless scale-out
- Serverless - auto-pause, pay for use
- Intelligent query processing + auto tuning

Azure SQL Managed Instance

- Near 100% SQL Server compatibility
- SQL Agent
- In-Memory OLTP (Business Critical tier)
- MI link for hybrid + disaster recovery

SQL database in Fabric

- OLTP + automatic analytics integration
- Auto-mirroring to OneLake Delta Parquet
- Automatic tuning (inherited from Azure SQL DB)
- Cross-database queries via three-part naming

Microsoft SQL – ground to cloud to fabric

The AI-ready enterprise database from ground to cloud

AZURE MEETUP PUGLIA

Develop once deploy anywhere

SQL Server 2025

Azure SQL

SQL database in Fabric

T-SQL

Developers

Engine

Tools

Fabric

AI

Copilots

#GLOBALAZURE 2026 · BY COMMUNITY · FOR COMMUNITY

global AI community

What problems are we trying to solve with AI?

- **Smarter searching** on your existing text data
- Bring in other text for **centralized vector searching**
- Provide **building blocks** - Intelligent assistants, RAG, AI Agents
- Take advantage of AI in a **secure and scalable** fashion
- **Overcome complexity** by using the familiar T-SQL language

SQL Server 2025

The AI-ready enterprise database from ground to cloud

AI built-in

Develop modern data applications

Integrate your data with Fabric

Secure by default

Mission critical engine

Connected with Arc

Managed DR

Assisted by Copilots

Benchmark[1] leader

Built for all platforms

1 All TPC Claims as of 11/18/2025 https://www.tpc.org/3408, https://www.tpc.org/3409, https://www.tpc.org/3410

SSMS 22.5

Update

aka.ms/ssms

- SQL Server 2025 support
- Fabric integration
- GitHub Copilot
- Windows ARM64 support
- Migration Assistant
- Query Hint Recommendation tool
- New themes including dark
- JSON viewer
- Zoom results grid

What are the features?

Build Agentic RAG patterns
***Inside* the engine**

Vector Store
Native vector data type and DiskANN index*

Model Management
Secure Model definitions ground or cloud

Embeddings built-in
Text Chunking and embedding generation

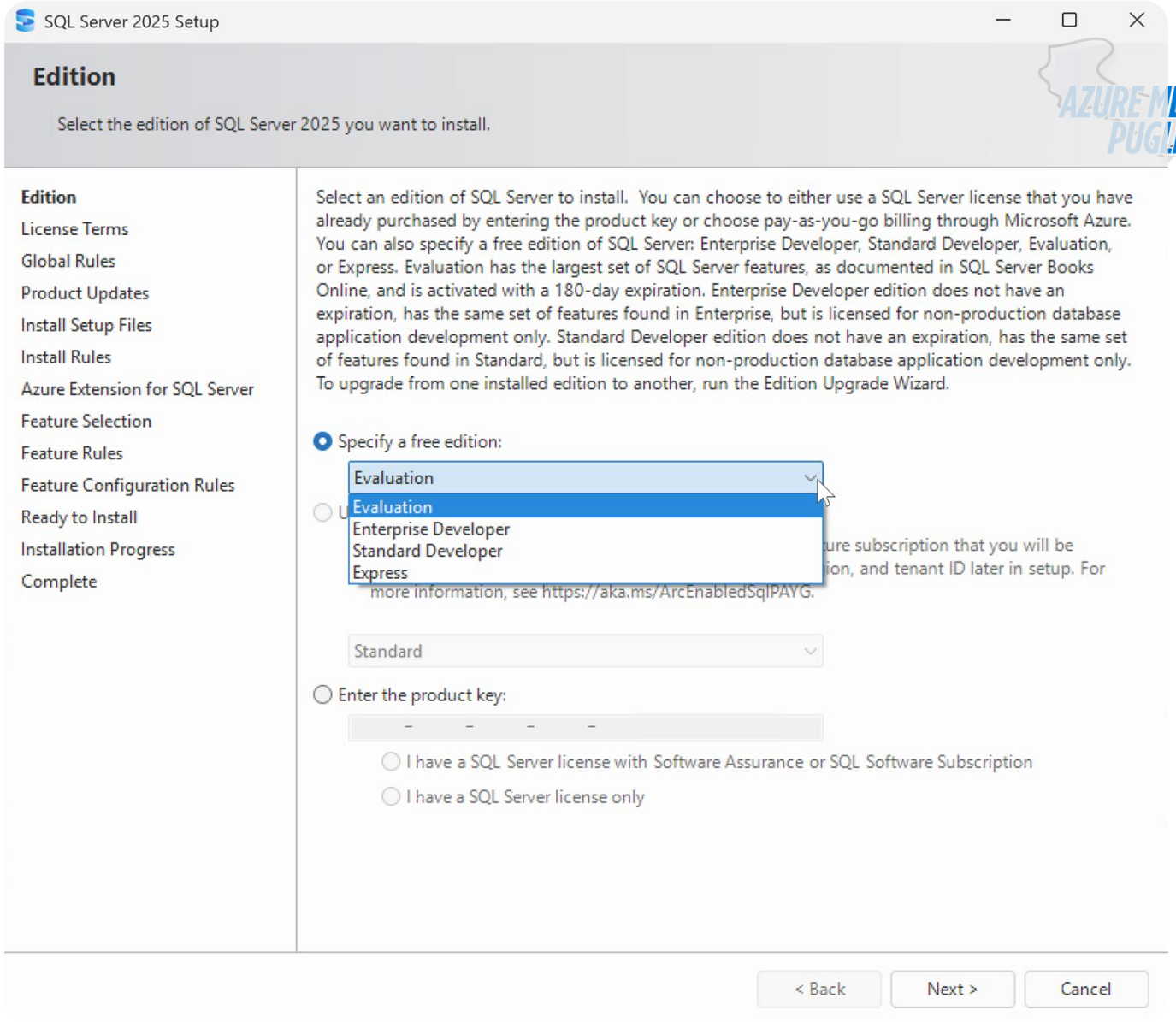
Simple semantic searching with hybrid
Vector distance (KNN) and Vector search (ANN)*

Framework integration
LangChain, Semantic Kernel, EF Core, Agent Framework

*Enabled by PREVIEW_FEATURES

Introducing Standard Developer Edition

- ✓ Free edition for dev and test purposes
- ✓ All features and limits from Standard Edition on all platforms
- ✓ Develop and test for Standard Edition with no license costs



SQL Server 2025 editions

The AI-ready enterprise database from ground to cloud

No pricing or licensing changes

Express

Free, entry-level database for small AI, web and mobile apps

Feature highlights

- 50 GB maximum relational databases size NEW
- Up to 4 cores of CPU
- Up to 1410 MBs of memory
- Built-in vector search and semantic search NEW
- Native JSON type support NEW
- External REST endpoint invocation NEW
- Change event streaming NEW
- Microsoft Entra ID authentication NEW
- Mirroring in Fabric NEW

Standard

Full featured database with extensive scale for mid-tier applications

Feature highlights

+ Express features

- Up to 32 cores of CPU NEW
- Up to 256 GB of memory NEW
- Optimized locking NEW
- Resource Governor NEW
- ZSTD Backup Compression Algorithm NEW
- Basic Availability Groups
- Azure SQL Managed Instance link
- Accelerated Database Recovery
- Backups to S3-compatible object storage
- 524 PB maximum relational databases size

Enterprise

Highest performance and scaling for business-critical workloads

Feature highlights

+ Express and Standard features

- Highest core count supported
- Highest memory supported
- Built-in Intelligent Query Processing
- Online and Resumable Index Operations
- Availability Groups: including distributed and contained
- Query Store for readable secondaries
- Persisted Statistics for Secondary Replicas (Query store)

Developer

Free to use for dev/test in non-production environments

Standard Developer NEW

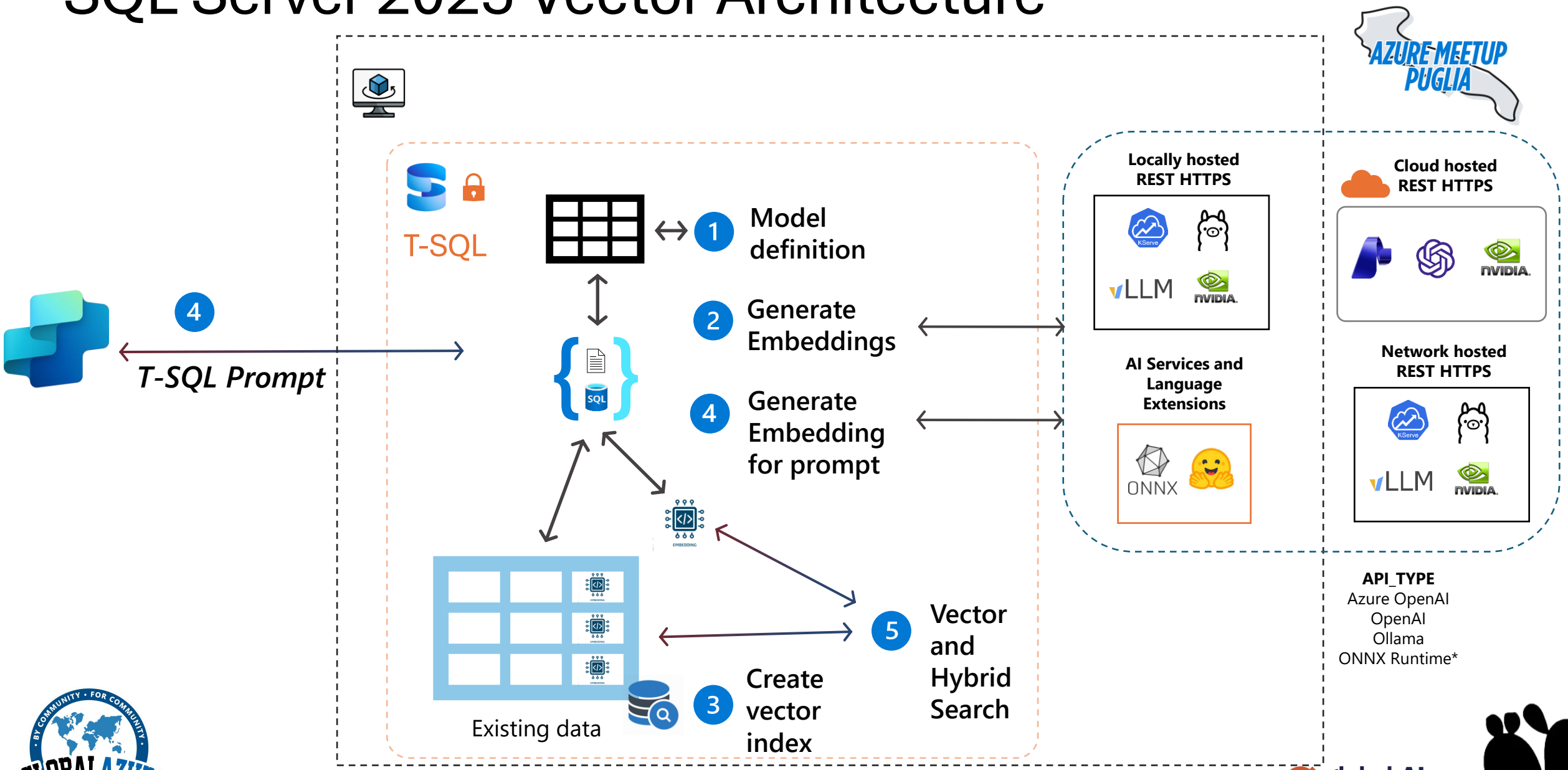
Enterprise Developer

A new way of searching with AI

How models interact with SQL data – Key concepts

- Tokens:
  - Text split into sub-word units
  - Cost scales per token
  - Affects model choice
- Embeddings:
  - Vectors representing semantic meaning
  - Vector (1536) type
  - Cosine similarity ranking
  - Similarity ranking
- Hybrid search:
  - Combine vector + WHERE clause
  - Single database query (no separate search needed)
  - Filter + rank combined

SQL Server 2025 Vector Architecture



External model setup workflow

Create a credential first:
Managed Identity (recommended) or API key authentication

Register the external model:
Database object with endpoint URL, format, and model name

Store vectors in VECTOR columns:
Native type with fixed dimensions (e.g. 1536 for text-embedding-3-small)

Generate with AI functions:
AI_GENERATE_EMBEDDINGS calls the model and returns vectors

embeddings_setup.sql

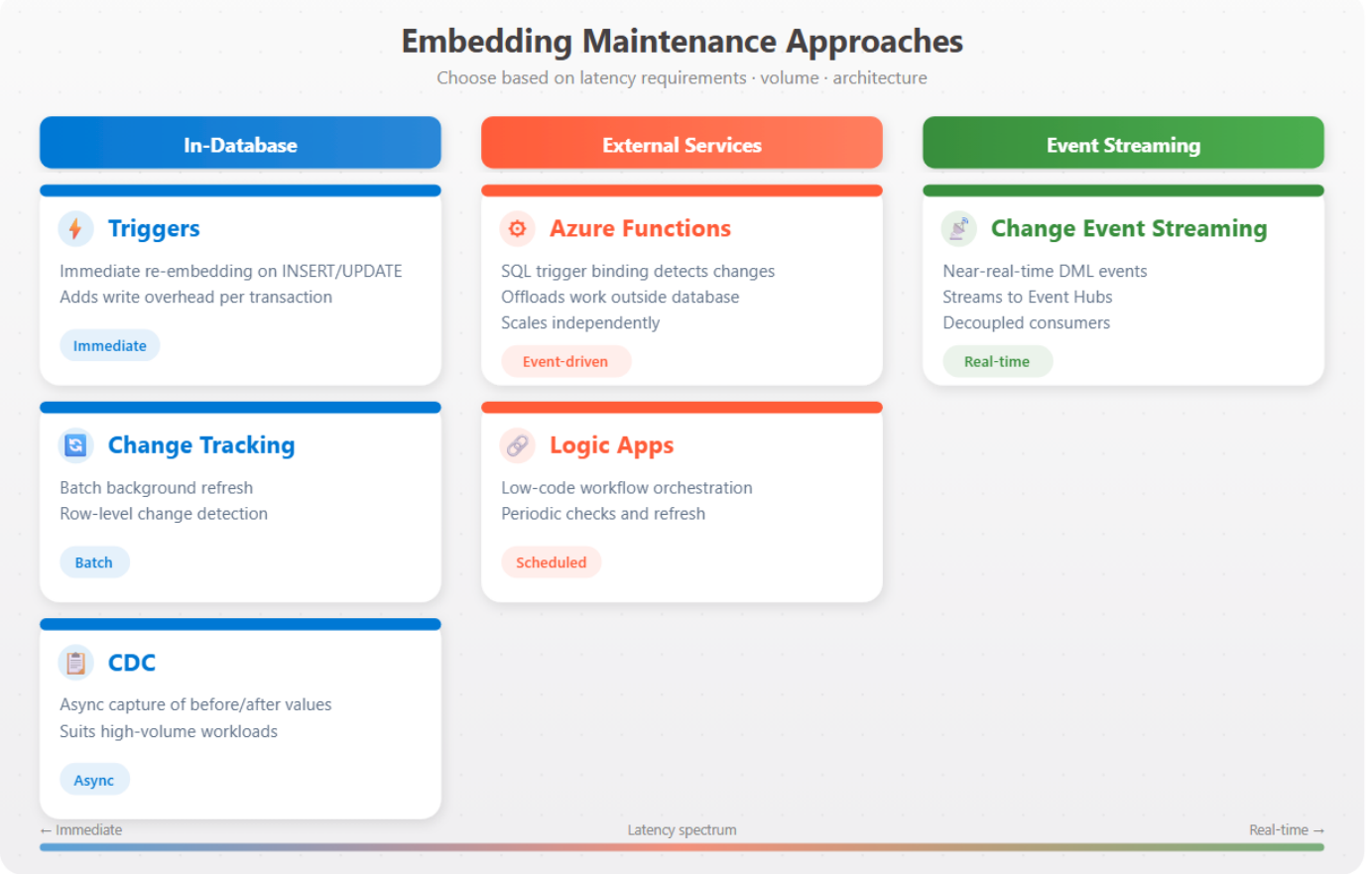
```sql
-- Step 1: Create a credential for managed identity
CREATE DATABASE SCOPED CREDENTIAL [https://<resource-name>.cognitiveservices.azure.com/]
    WITH IDENTITY = 'Managed Identity',
         SECRET   = '{"resourceid":"https://cognitiveservices.azure.com"}';

-- Step 2: Register the Azure OpenAI embedding model
CREATE EXTERNAL MODEL my_embedding_model
WITH (
    LOCATION   = 'https://<resource-name>.cognitiveservices.azure.com/openai/
                  deployments/<deployment-name>/embeddings?api-version=<api-version>',
    API_FORMAT = 'Azure OpenAI',
    MODEL_TYPE = EMBEDDINGS,
    MODEL      = 'text-embedding-3-small',
    CREDENTIAL = [https://<resource-name>.cognitiveservices.azure.com/]
);

-- Step 3: Create a table with a VECTOR column
CREATE TABLE dbo.documents (
    id        INT PRIMARY KEY,
    content   NVARCHAR(MAX),
    embedding VECTOR(1536)
);

-- Step 4: Generate embeddings from text
UPDATE dbo.documents
SET embedding = AI_GENERATE_EMBEDDINGS(content USE MODEL my_embedding_model);
```

External model setup — VECTOR storage — Generate embeddings

A new way of searching with AI
Demo

Generate and maintain embeddings

- **UPDATE SET embedding =** AI_GENERATE_EMBEDDINGS(...)
- **Batch initial load:** Then incremental refresh
- **VECTOR(1536) data type:** Efficient vector storage
- **Choose maintenance:** Based on volume, latency, architecture



Capturing changes with SQL Server

	Change Tracking (CT)	Change Data Capture (CDC)	Change Event Streaming (CES)
Capture Source	As part of committed transactions (sync)	Committed transactions from transaction log (async). Requires SQL Server Agent	Committed transactions from transaction log (async)
Capture Destination	System table (minimum)	System table (extra logging required)	Event stream (“IOless”)
Data Captured	Row identifier (DDL allowed)	Row identifier and data	Row identifier and data (DDL allowed)
Capture Method	Query system tables and user tables. Multiple destinations	Query system tables. Multiple destinations	Process event stream (“IOless”). Single destination
Availability	Immediate (pull-based)	Near real-time (pull-based)	Near real-time (push-based)
SQL effects	Transaction commit	Transaction log truncation .	Transaction log truncation
Typical Use Cases	Cache invalidation and sync	Data warehousing and auditing	Event-driven architectures – microservices integration, real-time analytics, cache sync, AI Agents

Capturing changes with SQL Server

```sql
-- Enable CES
EXEC sys.sp_enable_event_stream;
GO
```

```sql
-- Create the stream group
EXEC sys.sp_drop_event_stream_group N'OrdersCESGroup';
GO
EXEC sys.sp_create_event_stream_group
@stream_group_name = N'OrdersCESGroup',
@destination_type = N'AzureEventHubsAmqp',
@destination_location = N'sqleventhubspace.servicebus.windows.net/sqleventhub',
@destination_credential = eventhubscred,
@max_message_size_bytes = 1000000,
@partition_key_scheme = N'Default';
GO
EXEC sys.sp_add_object_to_event_stream_group N'OrdersCESGroup', N'dbo.Orders';
GO
```

Maintain embeddings
DEMO trigger

JSON

< SQL Server 2025

varchar/nvarchar column

Any index that supports varchar/nvarchar

Various functions for JSON

SQL Server 2025

json data type – binary storage up to **2GB**

.modify operator

json index

+ JSON_OBJECTAGG and JSON_ARRAYAGG

+ JSON_CONTAINS

+ JSON_QUERY WITH ARRAY WRAPPER

Save space with compressed storage
Less I/O and page reads

Regular expressions & fuzzy matching

Regex functions

- *REGEXP_LIKE*: filter rows by pattern
- *REGEXP_REPLACE*: clean/standardize text
- *REGEXP_SUBSTR*: extract matched substrings
- *REGEXP_COUNT* + *REGEXP_INSTR*
- *REGEXP_SPLIT_TO_TABLE*: split to rows

Fuzzy matching

- *EDIT_DISTANCE*: Levenshtein distance
- *EDIT_DISTANCE_SIMILARITY*: 0-100 score
- *JARO_WINKLER_DISTANCE*: name matching
- Prefix-weighted: better for short names
- Pre-filter with *LIKE* before fuzzy

Use cases

- Email/phone validation with regex
- Duplicate detection with fuzzy scores
- Data cleansing: standardize formats
- Typo-tolerant search (Jon ≈ John)
- Performance: pre-filter, then match

JSON – REGEXP
Demo

Coming Spring 2026

SQL AI Database Developer Certification

Level up and prove your skills

- Building AI-powered, LLM-integrated apps
- AI-augmented SQL development
- Advanced SQL Programming
- Security, governance, and responsible AI

aka.ms/learnSQLAI

Global Azure Puglia 2026 – 17 aprile

Build AI-powered solutions using SQL Server 2025

FEEDBACK

Grazie 😊

RESOURCES

https://techcommunity.microsoft.com/blog/coreinfrastructureandsecurityblog/ollama-on-https-for-sql-server/4458309

https://learn.microsoft.com/en-us/sql/t-sql/statements/create-json-index-transact-sql?view=sql-server-ver17

https://devblogs.microsoft.com/azure-sql/diskann-vector-index-improvements/

https://learn.microsoft.com/en-us/sql/t-sql/statements/create-vector-index-transact-sql?view=sql-server-ver17#migrating-from-earlier-vector-index-versions